JubileeOutlook Offline Support Implementation Guide

PostgreSQL Local Cache Architecture

Version: 1.0
Date: January 15, 2026
Author: Jubilee Development Team

Table of Contents

Overview

This document provides comprehensive instructions for implementing offline support in JubileeOutlook using PostgreSQL as the local cache database. The implementation enables:

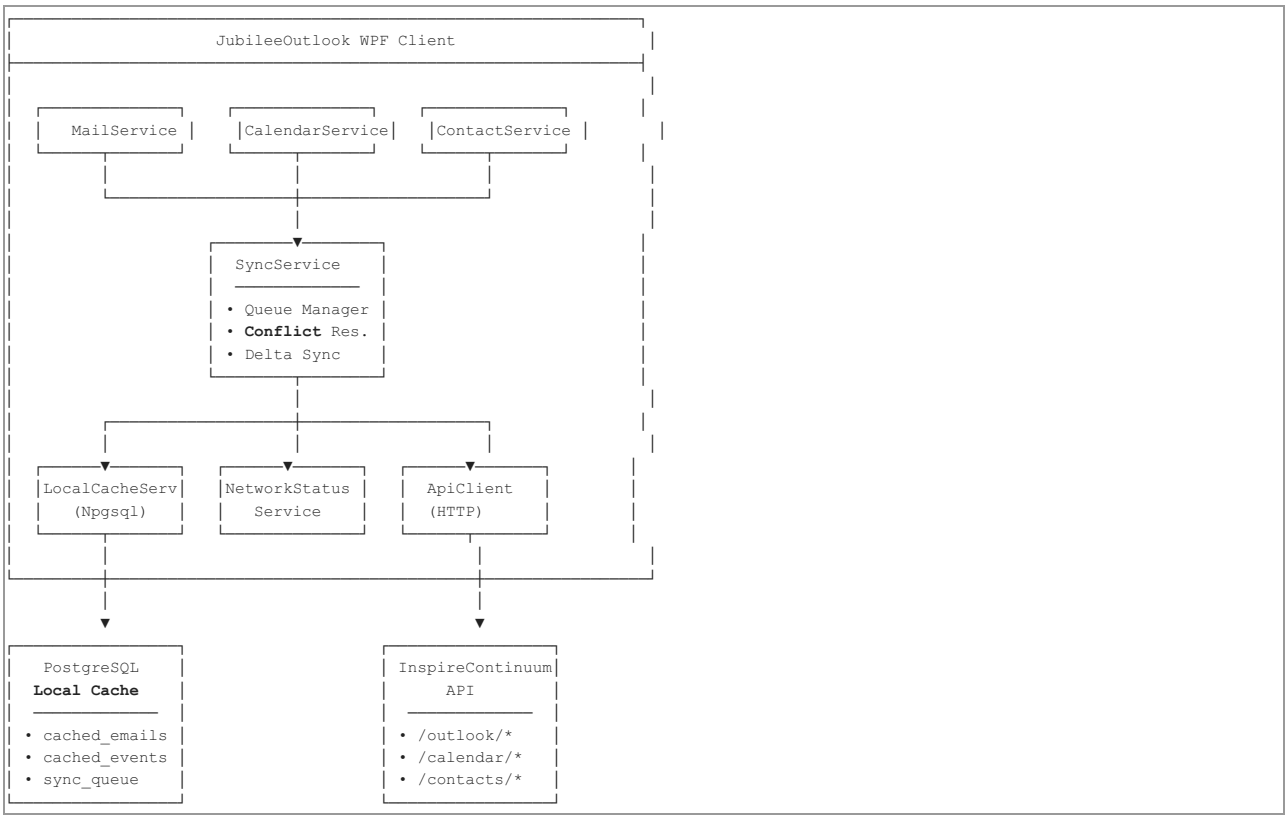
- **Offline Access**: Users can read emails, events, and contacts without network connectivity
- **Automatic Sync**: Changes sync automatically when connectivity is restored
- **Conflict Resolution**: Handles conflicts between local and server changes
- **Visual Indicators**: Clear UI feedback for online/offline status

Why PostgreSQL for Local Cache?

Feature	Benefit
Robust Concurrency	Handles multiple read/write operations safely
JSONB Support	Native JSON storage for flexible payload handling
Full SQL Support	Complex queries for sync operations
Consistent with Backend	Same database technology as InspireContinuum API
Enterprise Ready	Suitable for multi-user and networked scenarios

---

Architecture Diagram



Step 1: Add PostgreSQL NuGet Packages

Required Packages

Add the following NuGet packages to the JubileeOutlook.csproj:

```
<ItemGroup>
  <!-- PostgreSQL Driver -->
  <PackageReference Include="Npgsql" Version="8.0.1" />

  <!-- Optional: Entity Framework Core for PostgreSQL -->
  <PackageReference Include="Npgsql.EntityFrameworkCore.PostgreSQL" Version="8.0.0" />

  <!-- In-Memory Cache Layer -->
  <PackageReference Include="Microsoft.Extensions.Caching.Memory" Version="8.0.0" />

  <!-- JSON Serialization -->
  <PackageReference Include="System.Text.Json" Version="8.0.0" />
</ItemGroup>
```

Package Installation Commands

```
# Using .NET CLI
dotnet add package Npgsql --version 8.0.1
dotnet add package Microsoft.Extensions.Caching.Memory --version 8.0.0

# Using Package Manager Console
Install-Package Npgsql -Version 8.0.1
Install-Package Microsoft.Extensions.Caching.Memory -Version 8.0.0
```

Connection String Configuration

appsettings.json:

```
{
  "ConnectionStrings": {
    "LocalCache":
"Host=localhost;Port=5432;Database=jubilee_outlook_cache;Username=jubilee_app;Password=${POSTGRES_PASSWORD};Pooling=true;MinPoolSize=
1;MaxPoolSize=20"
  },
  "CacheSettings": {
    "EnableOfflineMode": true,
    "SyncIntervalSeconds": 300,
    "MaxCacheAgeDays": 30,
    "MaxSyncRetries": 3
  }
}
```

Environment Variables:

```
# Windows PowerShell
$env:JUBILEE_OUTLOOK_CACHE_DB =
"Host=localhost;Port=5432;Database=jubilee_outlook_cache;Username=jubilee_app;Password=your_secure_password"

# Windows CMD
set
JUBILEE_OUTLOOK_CACHE_DB=Host=localhost;Port=5432;Database=jubilee_outlook_cache;Username=jubilee_app;Password=your_secure_password

# Linux/macOS
export
JUBILEE_OUTLOOK_CACHE_DB="Host=localhost;Port=5432;Database=jubilee_outlook_cache;Username=jubilee_app;Password=your_secure_password"
```

Step 2: Create Local Database Schema

Database Creation Script

Run the following SQL script to create the cache database and all required tables:

```
-- ============================================================
-- JUBILEE OUTLOOK LOCAL CACHE DATABASE
-- PostgreSQL Schema Definition
-- Version: 1.0
-- ============================================================

-- Create the cache database (run as superuser)
CREATE DATABASE jubilee_outlook_cache
    WITH OWNER = jubilee_app
    ENCODING = 'UTF8'
    LC_COLLATE = 'en_US.UTF-8'
    LC_CTYPE = 'en_US.UTF-8'
    TEMPLATE = template0;

-- Connect to the new database
\c jubilee_outlook_cache

-- Enable required extensions
```

```sql
CREATE EXTENSION IF NOT EXISTS "uuid-ossp";
CREATE EXTENSION IF NOT EXISTS "pg_trgm";  -- For text search

-- ============================================================
-- EMAIL CACHE TABLES
-- ============================================================

-- Cached email folders
CREATE TABLE cached_email_folders (
    id UUID PRIMARY KEY DEFAULT uuid_generate_v4(),
    server_id VARCHAR(255) UNIQUE NOT NULL,
    user_id UUID NOT NULL,
    name VARCHAR(255) NOT NULL,
    folder_type VARCHAR(50) NOT NULL,
    parent_folder_id UUID REFERENCES cached_email_folders(id) ON DELETE CASCADE,
    icon VARCHAR(50),
    unread_count INTEGER DEFAULT 0,
    total_count INTEGER DEFAULT 0,
    is_system BOOLEAN DEFAULT FALSE,
    display_order INTEGER DEFAULT 0,

    -- Sync metadata
    last_synced_at TIMESTAMP WITH TIME ZONE,
    is_dirty BOOLEAN DEFAULT FALSE,
    sync_status VARCHAR(20) DEFAULT 'synced',
    sync_error TEXT,
    local_created_at TIMESTAMP WITH TIME ZONE DEFAULT CURRENT_TIMESTAMP,
    local_updated_at TIMESTAMP WITH TIME ZONE DEFAULT CURRENT_TIMESTAMP,

    CONSTRAINT chk_folder_type CHECK (folder_type IN ('inbox', 'sent', 'drafts', 'deleted', 'junk', 'archive', 'custom')),
    CONSTRAINT chk_sync_status CHECK (sync_status IN ('synced', 'pending_upload', 'pending_update', 'pending_delete', 'conflict',
'error'))
);

COMMENT ON TABLE cached_email_folders IS 'Local cache of email folders from InspireContinuum API';
COMMENT ON COLUMN cached_email_folders.server_id IS 'The ID from the server (InspireContinuum)';
COMMENT ON COLUMN cached_email_folders.is_dirty IS 'True if local changes need to be synced to server';

-- Cached email messages
CREATE TABLE cached_email_messages (
    id UUID PRIMARY KEY DEFAULT uuid_generate_v4(),
    server_id VARCHAR(255) UNIQUE,
    user_id UUID NOT NULL,
    folder_id UUID REFERENCES cached_email_folders(id) ON DELETE CASCADE,
    server_folder_id VARCHAR(255),

    -- Message content
    subject VARCHAR(1000),
    body_text TEXT,
    body_html TEXT,
    preview VARCHAR(500),
    sender_email VARCHAR(255),
    sender_name VARCHAR(255),

    -- Status flags
    is_read BOOLEAN DEFAULT FALSE,
    is_flagged BOOLEAN DEFAULT FALSE,
    is_draft BOOLEAN DEFAULT FALSE,
    has_attachments BOOLEAN DEFAULT FALSE,
    importance VARCHAR(20) DEFAULT 'normal',

    -- Conversation threading
    conversation_id VARCHAR(255),
    in_reply_to VARCHAR(255),

    -- Dates
    received_at TIMESTAMP WITH TIME ZONE,
    sent_at TIMESTAMP WITH TIME ZONE,

    -- Sync metadata
    last_synced_at TIMESTAMP WITH TIME ZONE,
    is_dirty BOOLEAN DEFAULT FALSE,
    dirty_fields TEXT[], -- Array of field names that have been modified locally
    sync_status VARCHAR(20) DEFAULT 'synced',
    sync_error TEXT,
    offline_created BOOLEAN DEFAULT FALSE,
    local_created_at TIMESTAMP WITH TIME ZONE DEFAULT CURRENT_TIMESTAMP,
    local_updated_at TIMESTAMP WITH TIME ZONE DEFAULT CURRENT_TIMESTAMP,

    CONSTRAINT chk_importance CHECK (importance IN ('low', 'normal', 'high')),
    CONSTRAINT chk_msg_sync_status CHECK (sync_status IN ('synced', 'pending_upload', 'pending_update', 'pending_delete', 'conflict',
'error'))
);

COMMENT ON TABLE cached_email_messages IS 'Local cache of email messages';
COMMENT ON COLUMN cached_email_messages.offline_created IS 'True if message was created while offline';
```

```
COMMENT ON COLUMN cached_email_messages.dirty_fields IS 'List of fields modified locally that need sync';

-- Cached email recipients
CREATE TABLE cached_email_recipients (
    id UUID PRIMARY KEY DEFAULT uuid_generate_v4(),
    message_id UUID NOT NULL REFERENCES cached_email_messages(id) ON DELETE CASCADE,
    email VARCHAR(255) NOT NULL,
    name VARCHAR(255),
    recipient_type VARCHAR(10) NOT NULL,
    display_order INTEGER DEFAULT 0,

    local_created_at TIMESTAMP WITH TIME ZONE DEFAULT CURRENT_TIMESTAMP,

    CONSTRAINT chk_recipient_type CHECK (recipient_type IN ('to', 'cc', 'bcc'))
);

COMMENT ON TABLE cached_email_recipients IS 'Recipients for cached email messages';

-- Cached email attachments
CREATE TABLE cached_email_attachments (
    id UUID PRIMARY KEY DEFAULT uuid_generate_v4(),
    server_id VARCHAR(255),
    message_id UUID NOT NULL REFERENCES cached_email_messages(id) ON DELETE CASCADE,
    file_name VARCHAR(500) NOT NULL,
    file_size BIGINT DEFAULT 0,
    mime_type VARCHAR(255),
    local_file_path VARCHAR(1000),
    is_cached BOOLEAN DEFAULT FALSE,
    is_inline BOOLEAN DEFAULT FALSE,
    content_id VARCHAR(255),

    local_created_at TIMESTAMP WITH TIME ZONE DEFAULT CURRENT_TIMESTAMP
);

COMMENT ON TABLE cached_email_attachments IS 'Attachments for cached email messages';
COMMENT ON COLUMN cached_email_attachments.local_file_path IS 'Path to locally cached attachment file';
COMMENT ON COLUMN cached_email_attachments.is_cached IS 'True if attachment content is cached locally';

-- ============================================================
-- CALENDAR CACHE TABLES
-- ============================================================

-- Cached calendars
CREATE TABLE cached_calendars (
    id UUID PRIMARY KEY DEFAULT uuid_generate_v4(),
    server_id VARCHAR(255) UNIQUE NOT NULL,
    user_id UUID NOT NULL,
    name VARCHAR(255) NOT NULL,
    description TEXT,
    color VARCHAR(20),
    is_default BOOLEAN DEFAULT FALSE,
    is_visible BOOLEAN DEFAULT TRUE,
    can_edit BOOLEAN DEFAULT TRUE,
    timezone VARCHAR(100) DEFAULT 'UTC',

    -- Sync metadata
    last_synced_at TIMESTAMP WITH TIME ZONE,
    is_dirty BOOLEAN DEFAULT FALSE,
    sync_status VARCHAR(20) DEFAULT 'synced',
    sync_error TEXT,
    local_created_at TIMESTAMP WITH TIME ZONE DEFAULT CURRENT_TIMESTAMP,
    local_updated_at TIMESTAMP WITH TIME ZONE DEFAULT CURRENT_TIMESTAMP,

    CONSTRAINT chk_cal_sync_status CHECK (sync_status IN ('synced', 'pending_upload', 'pending_update', 'pending_delete', 'conflict',
'error'))
);

COMMENT ON TABLE cached_calendars IS 'Local cache of calendars';

-- Cached calendar events
CREATE TABLE cached_calendar_events (
    id UUID PRIMARY KEY DEFAULT uuid_generate_v4(),
    server_id VARCHAR(255) UNIQUE,
    user_id UUID NOT NULL,
    calendar_id UUID REFERENCES cached_calendars(id) ON DELETE CASCADE,
    server_calendar_id VARCHAR(255),

    -- Event details
    title VARCHAR(500) NOT NULL,
    description TEXT,
    location VARCHAR(500),
    location_url VARCHAR(1000),

    -- Timing
    start_time TIMESTAMP WITH TIME ZONE NOT NULL,
    end_time TIMESTAMP WITH TIME ZONE NOT NULL,
```

```
    timezone VARCHAR(100) DEFAULT 'UTC',
    is_all_day BOOLEAN DEFAULT FALSE,

    -- Recurrence
    is_recurring BOOLEAN DEFAULT FALSE,
    recurrence_rule TEXT,
    recurrence_id VARCHAR(255),
    original_start_time TIMESTAMP WITH TIME ZONE,

    -- Status and reminders
    status VARCHAR(20) DEFAULT 'confirmed',
    show_as VARCHAR(20) DEFAULT 'busy',
    reminder_minutes INTEGER,

    -- Online meeting
    is_online_meeting BOOLEAN DEFAULT FALSE,
    online_meeting_url VARCHAR(1000),
    online_meeting_provider VARCHAR(50),

    -- Sync metadata
    last_synced_at TIMESTAMP WITH TIME ZONE,
    is_dirty BOOLEAN DEFAULT FALSE,
    dirty_fields TEXT[],
    sync_status VARCHAR(20) DEFAULT 'synced',
    sync_error TEXT,
    offline_created BOOLEAN DEFAULT FALSE,
    local_created_at TIMESTAMP WITH TIME ZONE DEFAULT CURRENT_TIMESTAMP,
    local_updated_at TIMESTAMP WITH TIME ZONE DEFAULT CURRENT_TIMESTAMP,

    CONSTRAINT chk_event_status CHECK (status IN ('confirmed', 'tentative', 'cancelled')),
    CONSTRAINT chk_show_as CHECK (show_as IN ('free', 'tentative', 'busy', 'oof', 'working_elsewhere')),
    CONSTRAINT chk_event_sync_status CHECK (sync_status IN ('synced', 'pending_upload', 'pending_update', 'pending_delete',
'conflict', 'error'))
);

COMMENT ON TABLE cached_calendar_events IS 'Local cache of calendar events';

-- Cached event attendees
CREATE TABLE cached_event_attendees (
    id UUID PRIMARY KEY DEFAULT uuid_generate_v4(),
    event_id UUID NOT NULL REFERENCES cached_calendar_events(id) ON DELETE CASCADE,
    email VARCHAR(255) NOT NULL,
    name VARCHAR(255),
    response_status VARCHAR(20) DEFAULT 'pending',
    is_organizer BOOLEAN DEFAULT FALSE,
    is_optional BOOLEAN DEFAULT FALSE,

    local_created_at TIMESTAMP WITH TIME ZONE DEFAULT CURRENT_TIMESTAMP,

    CONSTRAINT chk_response_status CHECK (response_status IN ('pending', 'accepted', 'declined', 'tentative'))
);

COMMENT ON TABLE cached_event_attendees IS 'Attendees for cached calendar events';

-- ============================================================
-- CONTACTS CACHE TABLES
-- ============================================================

-- Cached contacts
CREATE TABLE cached_contacts (
    id UUID PRIMARY KEY DEFAULT uuid_generate_v4(),
    server_id VARCHAR(255) UNIQUE,
    user_id UUID NOT NULL,

    -- Name fields
    display_name VARCHAR(255),
    first_name VARCHAR(100),
    middle_name VARCHAR(100),
    last_name VARCHAR(100),
    nickname VARCHAR(100),

    -- Contact info
    email_primary VARCHAR(255),
    email_secondary VARCHAR(255),
    email_other VARCHAR(255),
    phone_mobile VARCHAR(50),
    phone_work VARCHAR(50),
    phone_home VARCHAR(50),
    phone_other VARCHAR(50),

    -- Work info
    company VARCHAR(255),
    department VARCHAR(255),
    job_title VARCHAR(255),
    office_location VARCHAR(255),
```

```
    -- Address
    street_address VARCHAR(500),
    city VARCHAR(100),
    state VARCHAR(100),
    postal_code VARCHAR(20),
    country VARCHAR(100),

    -- Other
    birthday DATE,
    notes TEXT,
    photo_url VARCHAR(1000),
    local_photo_path VARCHAR(1000),

    -- Categorization
    categories TEXT[],
    is_favorite BOOLEAN DEFAULT FALSE,

    -- Sync metadata
    last_synced_at TIMESTAMP WITH TIME ZONE,
    is_dirty BOOLEAN DEFAULT FALSE,
    dirty_fields TEXT[],
    sync_status VARCHAR(20) DEFAULT 'synced',
    sync_error TEXT,
    offline_created BOOLEAN DEFAULT FALSE,
    local_created_at TIMESTAMP WITH TIME ZONE DEFAULT CURRENT_TIMESTAMP,
    local_updated_at TIMESTAMP WITH TIME ZONE DEFAULT CURRENT_TIMESTAMP,

    CONSTRAINT chk_contact_sync_status CHECK (sync_status IN ('synced', 'pending_upload', 'pending_update', 'pending_delete',
'conflict', 'error'))
);

COMMENT ON TABLE cached_contacts IS 'Local cache of contacts';

-- ============================================================
-- SYNC TRACKING TABLES
-- ============================================================

-- Sync operation queue for offline changes
CREATE TABLE sync_queue (
    id UUID PRIMARY KEY DEFAULT uuid_generate_v4(),
    user_id UUID NOT NULL,
    entity_type VARCHAR(50) NOT NULL,
    entity_id UUID NOT NULL,
    server_id VARCHAR(255),
    operation VARCHAR(20) NOT NULL,
    payload JSONB,

    -- Priority and ordering
    priority INTEGER DEFAULT 0,
    sequence_number SERIAL,

    -- Retry handling
    retry_count INTEGER DEFAULT 0,
    max_retries INTEGER DEFAULT 3,
    next_retry_at TIMESTAMP WITH TIME ZONE,

    -- Status tracking
    status VARCHAR(20) DEFAULT 'pending',
    error_message TEXT,
    error_code VARCHAR(50),

    -- Timestamps
    created_at TIMESTAMP WITH TIME ZONE DEFAULT CURRENT_TIMESTAMP,
    processing_started_at TIMESTAMP WITH TIME ZONE,
    processed_at TIMESTAMP WITH TIME ZONE,

    CONSTRAINT chk_entity_type CHECK (entity_type IN ('email', 'folder', 'event', 'calendar', 'contact', 'attachment')),
    CONSTRAINT chk_operation CHECK (operation IN ('create', 'update', 'delete', 'move', 'mark_read', 'flag')),
    CONSTRAINT chk_queue_status CHECK (status IN ('pending', 'processing', 'completed', 'failed', 'cancelled'))
);

COMMENT ON TABLE sync_queue IS 'Queue of pending sync operations for offline changes';
COMMENT ON COLUMN sync_queue.priority IS 'Higher values = higher priority (deletes > creates > updates)';
COMMENT ON COLUMN sync_queue.sequence_number IS 'Ensures operations are processed in order';

-- Sync state tracking per entity type
CREATE TABLE sync_state (
    id UUID PRIMARY KEY DEFAULT uuid_generate_v4(),
    user_id UUID NOT NULL,
    entity_type VARCHAR(50) NOT NULL,

    -- Sync tokens for delta sync
    last_sync_token VARCHAR(500),
    delta_link VARCHAR(1000),

    -- Sync timestamps
```

```
    last_full_sync_at TIMESTAMP WITH TIME ZONE,
    last_delta_sync_at TIMESTAMP WITH TIME ZONE,
    next_full_sync_at TIMESTAMP WITH TIME ZONE,

    -- Statistics
    total_synced_items INTEGER DEFAULT 0,
    sync_errors_count INTEGER DEFAULT 0,
    consecutive_failures INTEGER DEFAULT 0,

    -- Status
    sync_enabled BOOLEAN DEFAULT TRUE,
    last_error TEXT,

    created_at TIMESTAMP WITH TIME ZONE DEFAULT CURRENT_TIMESTAMP,
    updated_at TIMESTAMP WITH TIME ZONE DEFAULT CURRENT_TIMESTAMP,

    UNIQUE(user_id, entity_type)
);

COMMENT ON TABLE sync_state IS 'Tracks sync state for each entity type per user';
COMMENT ON COLUMN sync_state.last_sync_token IS 'Server-provided token for delta synchronization';

-- Conflict resolution log
CREATE TABLE sync_conflicts (
    id UUID PRIMARY KEY DEFAULT uuid_generate_v4(),
    user_id UUID NOT NULL,
    entity_type VARCHAR(50) NOT NULL,
    entity_id UUID NOT NULL,
    server_id VARCHAR(255),

    -- Conflict details
    conflict_type VARCHAR(50) NOT NULL,
    local_data JSONB,
    server_data JSONB,

    -- Resolution
    resolution VARCHAR(50),
    resolved_data JSONB,
    resolved_by VARCHAR(50), -- 'auto', 'user', 'server_wins', 'local_wins'
    resolved_at TIMESTAMP WITH TIME ZONE,

    created_at TIMESTAMP WITH TIME ZONE DEFAULT CURRENT_TIMESTAMP,

    CONSTRAINT chk_conflict_type CHECK (conflict_type IN ('update_conflict', 'delete_conflict', 'create_conflict')),
    CONSTRAINT chk_resolution CHECK (resolution IS NULL OR resolution IN ('server_wins', 'local_wins', 'merged', 'user_resolved'))
);

COMMENT ON TABLE sync_conflicts IS 'Log of sync conflicts for audit and debugging';

-- ============================================================
-- INDEXES FOR PERFORMANCE
-- ============================================================

-- Email indexes
CREATE INDEX idx_cached_messages_user_folder ON cached_email_messages(user_id, folder_id);
CREATE INDEX idx_cached_messages_server_id ON cached_email_messages(server_id) WHERE server_id IS NOT NULL;
CREATE INDEX idx_cached_messages_sync_status ON cached_email_messages(sync_status) WHERE sync_status != 'synced';
CREATE INDEX idx_cached_messages_is_dirty ON cached_email_messages(is_dirty) WHERE is_dirty = TRUE;
CREATE INDEX idx_cached_messages_received ON cached_email_messages(received_at DESC NULLS LAST);
CREATE INDEX idx_cached_messages_conversation ON cached_email_messages(conversation_id) WHERE conversation_id IS NOT NULL;
CREATE INDEX idx_cached_folders_user ON cached_email_folders(user_id);
CREATE INDEX idx_cached_folders_parent ON cached_email_folders(parent_folder_id) WHERE parent_folder_id IS NOT NULL;

-- Calendar indexes
CREATE INDEX idx_cached_events_user_calendar ON cached_calendar_events(user_id, calendar_id);
CREATE INDEX idx_cached_events_time_range ON cached_calendar_events(start_time, end_time);
CREATE INDEX idx_cached_events_sync_status ON cached_calendar_events(sync_status) WHERE sync_status != 'synced';
CREATE INDEX idx_cached_events_is_dirty ON cached_calendar_events(is_dirty) WHERE is_dirty = TRUE;
CREATE INDEX idx_cached_events_recurring ON cached_calendar_events(is_recurring) WHERE is_recurring = TRUE;
CREATE INDEX idx_cached_calendars_user ON cached_calendars(user_id);

-- Contact indexes
CREATE INDEX idx_cached_contacts_user ON cached_contacts(user_id);
CREATE INDEX idx_cached_contacts_email ON cached_contacts(email_primary);
CREATE INDEX idx_cached_contacts_name ON cached_contacts(display_name);
CREATE INDEX idx_cached_contacts_sync_status ON cached_contacts(sync_status) WHERE sync_status != 'synced';
CREATE INDEX idx_cached_contacts_favorite ON cached_contacts(is_favorite) WHERE is_favorite = TRUE;

-- Full-text search indexes
CREATE INDEX idx_cached_messages_subject_trgm ON cached_email_messages USING gin(subject gin_trgm_ops);
CREATE INDEX idx_cached_contacts_name_trgm ON cached_contacts USING gin(display_name gin_trgm_ops);

-- Sync queue indexes
CREATE INDEX idx_sync_queue_pending ON sync_queue(status, priority DESC, sequence_number) WHERE status = 'pending';
CREATE INDEX idx_sync_queue_user ON sync_queue(user_id, entity_type);
CREATE INDEX idx_sync_queue_retry ON sync_queue(next_retry_at) WHERE status = 'pending' AND retry_count > 0;
```

```sql
-- Sync state indexes
CREATE INDEX idx_sync_state_user ON sync_state(user_id);

-- ============================================================
-- TRIGGERS FOR AUTOMATIC TIMESTAMP UPDATES
-- ============================================================

-- Function to update local_updated_at timestamp
CREATE OR REPLACE FUNCTION update_local_updated_at()
RETURNS TRIGGER AS $
BEGIN
    NEW.local_updated_at = CURRENT_TIMESTAMP;
    RETURN NEW;
END;
$ LANGUAGE plpgsql;

-- Function to update sync_state updated_at
CREATE OR REPLACE FUNCTION update_sync_state_updated_at()
RETURNS TRIGGER AS $
BEGIN
    NEW.updated_at = CURRENT_TIMESTAMP;
    RETURN NEW;
END;
$ LANGUAGE plpgsql;

-- Apply triggers to all cached tables
CREATE TRIGGER tr_cached_messages_updated
    BEFORE UPDATE ON cached_email_messages
    FOR EACH ROW EXECUTE FUNCTION update_local_updated_at();

CREATE TRIGGER tr_cached_folders_updated
    BEFORE UPDATE ON cached_email_folders
    FOR EACH ROW EXECUTE FUNCTION update_local_updated_at();

CREATE TRIGGER tr_cached_events_updated
    BEFORE UPDATE ON cached_calendar_events
    FOR EACH ROW EXECUTE FUNCTION update_local_updated_at();

CREATE TRIGGER tr_cached_calendars_updated
    BEFORE UPDATE ON cached_calendars
    FOR EACH ROW EXECUTE FUNCTION update_local_updated_at();

CREATE TRIGGER tr_cached_contacts_updated
    BEFORE UPDATE ON cached_contacts
    FOR EACH ROW EXECUTE FUNCTION update_local_updated_at();

CREATE TRIGGER tr_sync_state_updated
    BEFORE UPDATE ON sync_state
    FOR EACH ROW EXECUTE FUNCTION update_sync_state_updated_at();

-- ============================================================
-- UTILITY FUNCTIONS
-- ============================================================

-- Function to clean up old sync queue entries
CREATE OR REPLACE FUNCTION cleanup_old_sync_queue(days_old INTEGER DEFAULT 30)
RETURNS INTEGER AS $
DECLARE
    deleted_count INTEGER;
BEGIN
    DELETE FROM sync_queue
    WHERE status IN ('completed', 'cancelled')
      AND processed_at < CURRENT_TIMESTAMP - (days_old || ' days')::INTERVAL;

    GET DIAGNOSTICS deleted_count = ROW_COUNT;
    RETURN deleted_count;
END;
$ LANGUAGE plpgsql;

-- Function to get pending sync count
CREATE OR REPLACE FUNCTION get_pending_sync_count(p_user_id UUID)
RETURNS TABLE(entity_type VARCHAR, pending_count BIGINT) AS $
BEGIN
    RETURN QUERY
    SELECT sq.entity_type, COUNT(*)
    FROM sync_queue sq
    WHERE sq.user_id = p_user_id AND sq.status = 'pending'
    GROUP BY sq.entity_type;
END;
$ LANGUAGE plpgsql;

-- Function to reset failed sync operations for retry
CREATE OR REPLACE FUNCTION reset_failed_sync_operations(p_user_id UUID)
RETURNS INTEGER AS $
DECLARE
```

```
    updated_count INTEGER;
BEGIN
    UPDATE sync_queue
    SET status = 'pending',
        retry_count = 0,
        error_message = NULL,
        next_retry_at = NULL
    WHERE user_id = p_user_id
      AND status = 'failed'
      AND retry_count < max_retries;

    GET DIAGNOSTICS updated_count = ROW_COUNT;
    RETURN updated_count;
END;
$ LANGUAGE plpgsql;

-- ============================================================
-- GRANTS (Adjust as needed for your environment)
-- ============================================================

-- Grant permissions to application user
GRANT ALL PRIVILEGES ON ALL TABLES IN SCHEMA public TO jubilee_app;
GRANT ALL PRIVILEGES ON ALL SEQUENCES IN SCHEMA public TO jubilee_app;
GRANT EXECUTE ON ALL FUNCTIONS IN SCHEMA public TO jubilee_app;
```

Step 3: Implement Cache Service

ILocalCacheService Interface

Create the interface that defines all cache operations:

```
// File: Services/ILocalCacheService.cs

using JubileeOutlook.Models;

namespace JubileeOutlook.Services;

/// <summary>
/// Interface for local PostgreSQL cache operations
/// Provides offline data access and sync queue management
/// </summary>
public interface ILocalCacheService
{
    #region Connection Management

    /// <summary>
    /// Initializes the cache service and verifies database connectivity
    /// </summary>
    Task<bool> InitializeAsync();

    /// <summary>
    /// Checks if the local cache database is available
    /// </summary>
    Task<bool> IsAvailableAsync();

    /// <summary>
    /// Gets the current cache statistics
    /// </summary>
    Task<CacheStatistics> GetStatisticsAsync();

    #endregion

    #region Email Caching

    /// <summary>
    /// Gets cached messages for a folder
    /// </summary>
    Task<List<EmailMessage>> GetCachedMessagesAsync(string folderId, int limit = 50, int offset = 0);

    /// <summary>
    /// Caches messages from the server
    /// </summary>
    Task CacheMessagesAsync(string folderId, List<EmailMessage> messages);

    /// <summary>
    /// Gets a single cached message by ID
    /// </summary>
    Task<EmailMessage?> GetCachedMessageAsync(string messageId);

    /// <summary>
    /// Caches or updates a single message
    /// </summary>
    Task CacheMessageAsync(EmailMessage message);

    /// <summary>
```

```
    /// Marks a message as modified locally (dirty)
    /// </summary>
    Task MarkMessageDirtyAsync(string messageId, string operation, string[]? dirtyFields = null);

    /// <summary>
    /// Gets all dirty messages that need syncing
    /// </summary>
    Task<List<EmailMessage>> GetDirtyMessagesAsync();

    #endregion

    #region Folder Caching

    /// <summary>
    /// Gets cached email folders
    /// </summary>
    Task<List<MailFolder>> GetCachedFoldersAsync();

    /// <summary>
    /// Caches folders from the server
    /// </summary>
    Task CacheFoldersAsync(List<MailFolder> folders);

    #endregion

    #region Calendar Caching

    /// <summary>
    /// Gets cached events for a date range
    /// </summary>
    Task<List<CalendarEvent>> GetCachedEventsAsync(DateTime startDate, DateTime endDate);

    /// <summary>
    /// Caches events from the server
    /// </summary>
    Task CacheEventsAsync(List<CalendarEvent> events);

    /// <summary>
    /// Gets a single cached event by ID
    /// </summary>
    Task<CalendarEvent?> GetCachedEventAsync(string eventId);

    /// <summary>
    /// Caches or updates a single event
    /// </summary>
    Task CacheEventAsync(CalendarEvent calendarEvent);

    #endregion

    #region Contact Caching

    /// <summary>
    /// Gets cached contacts
    /// </summary>
    Task<List<Contact>> GetCachedContactsAsync(string? searchQuery = null);

    /// <summary>
    /// Caches contacts from the server
    /// </summary>
    Task CacheContactsAsync(List<Contact> contacts);

    #endregion

    #region Sync Queue Management

    /// <summary>
    /// Queues an operation for sync when back online
    /// </summary>
    Task QueueSyncOperationAsync(string entityType, Guid entityId, string operation, object payload, int priority = 0);

    /// <summary>
    /// Gets pending sync operations
    /// </summary>
    Task<List<SyncQueueItem>> GetPendingSyncOperationsAsync(int limit = 50);

    /// <summary>
    /// Marks a sync operation as completed
    /// </summary>
    Task MarkSyncOperationCompletedAsync(Guid operationId);

    /// <summary>
    /// Marks a sync operation as failed
    /// </summary>
    Task MarkSyncOperationFailedAsync(Guid operationId, string errorMessage, string? errorCode = null);

    /// <summary>
```

```
    /// Gets the count of pending sync operations
    /// </summary>
    Task<int> GetPendingSyncCountAsync();

    #endregion

    #region Sync State Management

    /// <summary>
    /// Gets the sync state for an entity type
    /// </summary>
    Task<SyncState?> GetSyncStateAsync(string entityType);

    /// <summary>
    /// Updates the sync state after a successful sync
    /// </summary>
    Task UpdateSyncStateAsync(string entityType, string? syncToken, DateTime syncTime);

    #endregion

    #region Cache Maintenance

    /// <summary>
    /// Invalidates cache for a specific entity type
    /// </summary>
    Task InvalidateCacheAsync(string entityType);

    /// <summary>
    /// Clears all cached data
    /// </summary>
    Task ClearAllCacheAsync();

    /// <summary>
    /// Removes old cached data beyond retention period
    /// </summary>
    Task PruneCacheAsync(int maxAgeDays = 30);

    #endregion
}
```

LocalCacheService Implementation

```
// File: Services/LocalCacheService.cs

using Npgsql;
using System.Text.Json;
using JubileeOutlook.Models;

namespace JubileeOutlook.Services;

/// <summary>
/// PostgreSQL-based local cache service for offline support
/// </summary>
public class LocalCacheService : ILocalCacheService, IDisposable
{
    private readonly string _connectionString;
    private readonly JsonSerializerOptions _jsonOptions;
    private NpgsqlDataSource? _dataSource;
    private bool _isInitialized = false;
    private bool _disposed = false;

    public LocalCacheService()
    {
        _connectionString = GetConnectionString();

        _jsonOptions = new JsonSerializerOptions
        {
            PropertyNamingPolicy = JsonNamingPolicy.CamelCase,
            WriteIndented = false,
            DefaultIgnoreCondition = System.Text.Json.Serialization.JsonIgnoreCondition.WhenWritingNull
        };
    }

    private static string GetConnectionString()
    {
        // Priority: Environment variable > Config file
        var envConnString = Environment.GetEnvironmentVariable("JUBILEE_OUTLOOK_CACHE_DB");
        if (!string.IsNullOrEmpty(envConnString))
        {
            return envConnString;
        }

        // Fallback to configuration
        var config = ConfigurationService.Instance;
        return config.ConnectionStrings?.LocalCache
            ?? "Host=localhost;Port=5432;Database=jubilee_outlook_cache;Username=jubilee_app;Password=password";
```

```
}

#region Connection Management

public async Task<bool> InitializeAsync()
{
    try
    {
        var dataSourceBuilder = new NpgsqlDataSourceBuilder(_connectionString);
        _dataSource = dataSourceBuilder.Build();

        // Test connection
        await using var conn = await _dataSource.OpenConnectionAsync();
        await using var cmd = new NpgsqlCommand("SELECT 1", conn);
        await cmd.ExecuteScalarAsync();

        _isInitialized = true;
        System.Diagnostics.Debug.WriteLine("[LocalCacheService] PostgreSQL cache initialized successfully");
        return true;
    }
    catch (Exception ex)
    {
        System.Diagnostics.Debug.WriteLine({{content}}quot;[LocalCacheService] Failed to initialize: {ex.Message}");
        _isInitialized = false;
        return false;
    }
}

public async Task<bool> IsAvailableAsync()
{
    if (!_isInitialized || _dataSource == null) return false;

    try
    {
        await using var conn = await _dataSource.OpenConnectionAsync();
        return true;
    }
    catch
    {
        return false;
    }
}

public async Task<CacheStatistics> GetStatisticsAsync()
{
    var stats = new CacheStatistics();

    if (!_isInitialized || _dataSource == null) return stats;

    try
    {
        await using var conn = await _dataSource.OpenConnectionAsync();

        // Get counts for each entity type
        await using var cmd = new NpgsqlCommand(@"
            SELECT
                (SELECT COUNT(*) FROM cached_email_messages) as email_count,
                (SELECT COUNT(*) FROM cached_email_folders) as folder_count,
                (SELECT COUNT(*) FROM cached_calendar_events) as event_count,
                (SELECT COUNT(*) FROM cached_contacts) as contact_count,
                (SELECT COUNT(*) FROM sync_queue WHERE status = 'pending') as pending_sync_count
        ", conn);

        await using var reader = await cmd.ExecuteReaderAsync();
        if (await reader.ReadAsync())
        {
            stats.CachedEmailCount = reader.GetInt64(0);
            stats.CachedFolderCount = reader.GetInt64(1);
            stats.CachedEventCount = reader.GetInt64(2);
            stats.CachedContactCount = reader.GetInt64(3);
            stats.PendingSyncCount = reader.GetInt64(4);
        }
    }
    catch (Exception ex)
    {
        System.Diagnostics.Debug.WriteLine({{content}}quot;[LocalCacheService] GetStatistics error: {ex.Message}");
    }

    return stats;
}

#endregion

#region Email Caching

public async Task<List<EmailMessage>> GetCachedMessagesAsync(string folderId, int limit = 50, int offset = 0)
```

```
    {
        var messages = new List<EmailMessage>();

        if (!_isInitialized || _dataSource == null) return messages;

        try
        {
            await using var conn = await _dataSource.OpenConnectionAsync();

            await using var cmd = new NpgsqlCommand(@"
                SELECT m.id, m.server_id, m.subject, m.body_text, m.body_html, m.preview,
                       m.sender_email, m.sender_name, m.is_read, m.is_flagged, m.is_draft,
                       m.has_attachments, m.importance, m.received_at, m.sent_at,
                       m.conversation_id, m.sync_status
                FROM cached_email_messages m
                WHERE m.server_folder_id = @folderId
                ORDER BY m.received_at DESC NULLS LAST
                LIMIT @limit OFFSET @offset
            ", conn);

            cmd.Parameters.AddWithValue("folderId", folderId);
            cmd.Parameters.AddWithValue("limit", limit);
            cmd.Parameters.AddWithValue("offset", offset);

            await using var reader = await cmd.ExecuteReaderAsync();
            while (await reader.ReadAsync())
            {
                messages.Add(MapToEmailMessage(reader));
            }

            // Load recipients for each message
            foreach (var message in messages)
            {
                message.To = await GetMessageRecipientsAsync(conn, message.Id, "to");
                message.Cc = await GetMessageRecipientsAsync(conn, message.Id, "cc");
                message.Bcc = await GetMessageRecipientsAsync(conn, message.Id, "bcc");
            }

            System.Diagnostics.Debug.WriteLine(&{{content}}quot;[LocalCacheService] Retrieved {messages.Count} cached messages for
folder {folderId}");
        }
        catch (Exception ex)
        {
            System.Diagnostics.Debug.WriteLine(&{{content}}quot;[LocalCacheService] GetCachedMessages error: {ex.Message}");
        }

        return messages;
    }

    private async Task<List<string>> GetMessageRecipientsAsync(NpgsqlConnection conn, string messageId, string recipientType)
    {
        var recipients = new List<string>();

        await using var cmd = new NpgsqlCommand(@"
            SELECT email FROM cached_email_recipients
            WHERE message_id = (SELECT id FROM cached_email_messages WHERE server_id = @messageId)
              AND recipient_type = @type
            ORDER BY display_order
        ", conn);

        cmd.Parameters.AddWithValue("messageId", messageId);
        cmd.Parameters.AddWithValue("type", recipientType);

        await using var reader = await cmd.ExecuteReaderAsync();
        while (await reader.ReadAsync())
        {
            recipients.Add(reader.GetString(0));
        }

        return recipients;
    }

    public async Task CacheMessagesAsync(string folderId, List<EmailMessage> messages)
    {
        if (!_isInitialized || _dataSource == null) return;

        try
        {
            await using var conn = await _dataSource.OpenConnectionAsync();
            await using var transaction = await conn.BeginTransactionAsync();

            try
            {
                foreach (var message in messages)
                {
                    await UpsertMessageAsync(conn, folderId, message);
```

```
                }

                await transaction.CommitAsync();
                System.Diagnostics.Debug.WriteLine({{content}}quot;[LocalCacheService] Cached {messages.Count} messages for folder
{folderId}");
            }
            catch
            {
                await transaction.RollbackAsync();
                throw;
            }
        }
        catch (Exception ex)
        {
            System.Diagnostics.Debug.WriteLine({{content}}quot;[LocalCacheService] CacheMessages error: {ex.Message}");
        }
    }

    private async Task UpsertMessageAsync(NpgsqlConnection conn, string folderId, EmailMessage message)
    {
        // Get or create the user_id (using service configuration)
        var userId = Guid.TryParse(ServiceConfiguration.UserId, out var uid) ? uid : Guid.Empty;

        await using var cmd = new NpgsqlCommand(@"
            INSERT INTO cached_email_messages
                (server_id, user_id, server_folder_id, subject, body_text, body_html, preview,
                 sender_email, sender_name, is_read, is_flagged, is_draft,
                 has_attachments, importance, received_at, sent_at, conversation_id, last_synced_at)
            VALUES
                (@serverId, @userId, @folderId, @subject, @bodyText, @bodyHtml, @preview,
                 @senderEmail, @senderName, @isRead, @isFlagged, @isDraft,
                 @hasAttachments, @importance, @receivedAt, @sentAt, @conversationId, @lastSynced)
            ON CONFLICT (server_id) DO UPDATE SET
                subject = EXCLUDED.subject,
                body_text = EXCLUDED.body_text,
                body_html = EXCLUDED.body_html,
                preview = EXCLUDED.preview,
                sender_email = EXCLUDED.sender_email,
                sender_name = EXCLUDED.sender_name,
                is_read = EXCLUDED.is_read,
                is_flagged = EXCLUDED.is_flagged,
                has_attachments = EXCLUDED.has_attachments,
                importance = EXCLUDED.importance,
                last_synced_at = EXCLUDED.last_synced_at
            WHERE cached_email_messages.sync_status = 'synced'
        ", conn);

        cmd.Parameters.AddWithValue("serverId", message.Id);
        cmd.Parameters.AddWithValue("userId", userId);
        cmd.Parameters.AddWithValue("folderId", folderId);
        cmd.Parameters.AddWithValue("subject", message.Subject ?? "");
        cmd.Parameters.AddWithValue("bodyText", message.IsHtml ? (object)DBNull.Value : message.Body ?? "");
        cmd.Parameters.AddWithValue("bodyHtml", message.IsHtml ? message.Body ?? "" : (object)DBNull.Value);
        cmd.Parameters.AddWithValue("preview", message.Preview ?? "");
        cmd.Parameters.AddWithValue("senderEmail", message.FromEmail ?? "");
        cmd.Parameters.AddWithValue("senderName", message.From ?? "");
        cmd.Parameters.AddWithValue("isRead", message.IsRead);
        cmd.Parameters.AddWithValue("isFlagged", message.IsFlagged);
        cmd.Parameters.AddWithValue("isDraft", false);
        cmd.Parameters.AddWithValue("hasAttachments", message.HasAttachments);
        cmd.Parameters.AddWithValue("importance", message.Priority.ToString().ToLower());
        cmd.Parameters.AddWithValue("receivedAt", message.ReceivedDate);
        cmd.Parameters.AddWithValue("sentAt", message.SentDate);
        cmd.Parameters.AddWithValue("conversationId", message.ConversationId ?? (object)DBNull.Value);
        cmd.Parameters.AddWithValue("lastSynced", DateTime.UtcNow);

        await cmd.ExecuteNonQueryAsync();

        // Cache recipients
        await CacheMessageRecipientsAsync(conn, message);
    }

    private async Task CacheMessageRecipientsAsync(NpgsqlConnection conn, EmailMessage message)
    {
        // Delete existing recipients
        await using var deleteCmd = new NpgsqlCommand(@"
            DELETE FROM cached_email_recipients
            WHERE message_id = (SELECT id FROM cached_email_messages WHERE server_id = @messageId)
        ", conn);
        deleteCmd.Parameters.AddWithValue("messageId", message.Id);
        await deleteCmd.ExecuteNonQueryAsync();

        // Insert new recipients
        var order = 0;
        foreach (var email in message.To ?? new List<string>())
        {
```

```
            await InsertRecipientAsync(conn, message.Id, email, "to", order++);
        }
        order = 0;
        foreach (var email in message.Cc ?? new List<string>())
        {
            await InsertRecipientAsync(conn, message.Id, email, "cc", order++);
        }
        order = 0;
        foreach (var email in message.Bcc ?? new List<string>())
        {
            await InsertRecipientAsync(conn, message.Id, email, "bcc", order++);
        }
    }

    private async Task InsertRecipientAsync(NpgsqlConnection conn, string messageId, string email, string type, int order)
    {
        await using var cmd = new NpgsqlCommand(@"
            INSERT INTO cached_email_recipients (message_id, email, recipient_type, display_order)
            SELECT id, @email, @type, @order
            FROM cached_email_messages WHERE server_id = @messageId
        ", conn);

        cmd.Parameters.AddWithValue("messageId", messageId);
        cmd.Parameters.AddWithValue("email", email);
        cmd.Parameters.AddWithValue("type", type);
        cmd.Parameters.AddWithValue("order", order);

        await cmd.ExecuteNonQueryAsync();
    }

    public async Task<EmailMessage?> GetCachedMessageAsync(string messageId)
    {
        if (!_isInitialized || _dataSource == null) return null;

        try
        {
            await using var conn = await _dataSource.OpenConnectionAsync();

            await using var cmd = new NpgsqlCommand(@"
                SELECT id, server_id, subject, body_text, body_html, preview,
                       sender_email, sender_name, is_read, is_flagged, is_draft,
                       has_attachments, importance, received_at, sent_at,
                       conversation_id, sync_status
                FROM cached_email_messages
                WHERE server_id = @messageId
            ", conn);

            cmd.Parameters.AddWithValue("messageId", messageId);

            await using var reader = await cmd.ExecuteReaderAsync();
            if (await reader.ReadAsync())
            {
                var message = MapToEmailMessage(reader);

                // Load recipients
                await reader.CloseAsync();
                message.To = await GetMessageRecipientsAsync(conn, messageId, "to");
                message.Cc = await GetMessageRecipientsAsync(conn, messageId, "cc");
                message.Bcc = await GetMessageRecipientsAsync(conn, messageId, "bcc");

                return message;
            }
        }
        catch (Exception ex)
        {
            System.Diagnostics.Debug.WriteLine({{content}}quot;[LocalCacheService] GetCachedMessage error: {ex.Message}");
        }

        return null;
    }

    public async Task CacheMessageAsync(EmailMessage message)
    {
        if (!_isInitialized || _dataSource == null) return;

        try
        {
            await using var conn = await _dataSource.OpenConnectionAsync();
            await UpsertMessageAsync(conn, message.FolderId, message);
        }
        catch (Exception ex)
        {
            System.Diagnostics.Debug.WriteLine({{content}}quot;[LocalCacheService] CacheMessage error: {ex.Message}");
        }
    }
```

```
public async Task MarkMessageDirtyAsync(string messageId, string operation, string[]? dirtyFields = null)
{
    if (!_isInitialized || _dataSource == null) return;

    try
    {
        await using var conn = await _dataSource.OpenConnectionAsync();

        await using var cmd = new NpgsqlCommand(@"
            UPDATE cached_email_messages
            SET is_dirty = TRUE,
                dirty_fields = @dirtyFields,
                sync_status = @syncStatus
            WHERE server_id = @messageId
        ", conn);

        cmd.Parameters.AddWithValue("messageId", messageId);
        cmd.Parameters.AddWithValue("dirtyFields", dirtyFields ?? Array.Empty<string>());
        cmd.Parameters.AddWithValue("syncStatus", operation == "delete" ? "pending_delete" : "pending_update");

        await cmd.ExecuteNonQueryAsync();
    }
    catch (Exception ex)
    {
        System.Diagnostics.Debug.WriteLine({{content}}quot;[LocalCacheService] MarkMessageDirty error: {ex.Message}");
    }
}

public async Task<List<EmailMessage>> GetDirtyMessagesAsync()
{
    var messages = new List<EmailMessage>();

    if (!_isInitialized || _dataSource == null) return messages;

    try
    {
        await using var conn = await _dataSource.OpenConnectionAsync();

        await using var cmd = new NpgsqlCommand(@"
            SELECT id, server_id, subject, body_text, body_html, preview,
                   sender_email, sender_name, is_read, is_flagged, is_draft,
                   has_attachments, importance, received_at, sent_at,
                   conversation_id, sync_status
            FROM cached_email_messages
            WHERE is_dirty = TRUE
            ORDER BY local_updated_at
        ", conn);

        await using var reader = await cmd.ExecuteReaderAsync();
        while (await reader.ReadAsync())
        {
            messages.Add(MapToEmailMessage(reader));
        }
    }
    catch (Exception ex)
    {
        System.Diagnostics.Debug.WriteLine({{content}}quot;[LocalCacheService] GetDirtyMessages error: {ex.Message}");
    }

    return messages;
}

private EmailMessage MapToEmailMessage(NpgsqlDataReader reader)
{
    return new EmailMessage
    {
        Id = reader.IsDBNull(1) ? reader.GetGuid(0).ToString() : reader.GetString(1),
        Subject = reader.IsDBNull(2) ? "" : reader.GetString(2),
        Body = reader.IsDBNull(3) ? (reader.IsDBNull(4) ? "" : reader.GetString(4)) : reader.GetString(3),
        IsHtml = !reader.IsDBNull(4),
        Preview = reader.IsDBNull(5) ? "" : reader.GetString(5),
        FromEmail = reader.IsDBNull(6) ? "" : reader.GetString(6),
        From = reader.IsDBNull(7) ? "" : reader.GetString(7),
        IsRead = reader.GetBoolean(8),
        IsFlagged = reader.GetBoolean(9),
        HasAttachments = reader.GetBoolean(11),
        Priority = ParsePriority(reader.IsDBNull(12) ? "normal" : reader.GetString(12)),
        ReceivedDate = reader.IsDBNull(13) ? DateTime.Now : reader.GetDateTime(13),
        SentDate = reader.IsDBNull(14) ? DateTime.Now : reader.GetDateTime(14),
        ConversationId = reader.IsDBNull(15) ? "" : reader.GetString(15)
    };
}

private static EmailPriority ParsePriority(string priority)
{
    return priority.ToLower() switch
```

```
    {
        "high" => EmailPriority.High,
        "low" => EmailPriority.Low,
        _ => EmailPriority.Normal
    };
}

#endregion

#region Sync Queue Management

public async Task QueueSyncOperationAsync(string entityType, Guid entityId, string operation, object payload, int priority = 0)
{
    if (!_isInitialized || _dataSource == null) return;

    try
    {
        var userId = Guid.TryParse(ServiceConfiguration.UserId, out var uid) ? uid : Guid.Empty;
        var payloadJson = JsonSerializer.Serialize(payload, _jsonOptions);

        await using var conn = await _dataSource.OpenConnectionAsync();

        await using var cmd = new NpgsqlCommand(@"
            INSERT INTO sync_queue (user_id, entity_type, entity_id, operation, payload, priority)
            VALUES (@userId, @entityType, @entityId, @operation, @payload::jsonb, @priority)
        ", conn);

        cmd.Parameters.AddWithValue("userId", userId);
        cmd.Parameters.AddWithValue("entityType", entityType);
        cmd.Parameters.AddWithValue("entityId", entityId);
        cmd.Parameters.AddWithValue("operation", operation);
        cmd.Parameters.AddWithValue("payload", payloadJson);
        cmd.Parameters.AddWithValue("priority", priority);

        await cmd.ExecuteNonQueryAsync();

        System.Diagnostics.Debug.WriteLine(&quot;[LocalCacheService] Queued {operation} for {entityType} {entityId}");
    }
    catch (Exception ex)
    {
        System.Diagnostics.Debug.WriteLine(&quot;[LocalCacheService] QueueSyncOperation error: {ex.Message}");
    }
}

public async Task<List<SyncQueueItem>> GetPendingSyncOperationsAsync(int limit = 50)
{
    var items = new List<SyncQueueItem>();

    if (!_isInitialized || _dataSource == null) return items;

    try
    {
        await using var conn = await _dataSource.OpenConnectionAsync();

        await using var cmd = new NpgsqlCommand(@"
            SELECT id, entity_type, entity_id, server_id, operation, payload, retry_count, created_at
            FROM sync_queue
            WHERE status = 'pending'
              AND retry_count < max_retries
              AND (next_retry_at IS NULL OR next_retry_at <= CURRENT_TIMESTAMP)
            ORDER BY priority DESC, sequence_number ASC
            LIMIT @limit
        ", conn);

        cmd.Parameters.AddWithValue("limit", limit);

        await using var reader = await cmd.ExecuteReaderAsync();
        while (await reader.ReadAsync())
        {
            items.Add(new SyncQueueItem
            {
                Id = reader.GetGuid(0),
                EntityType = reader.GetString(1),
                EntityId = reader.GetGuid(2),
                ServerId = reader.IsDBNull(3) ? null : reader.GetString(3),
                Operation = reader.GetString(4),
                Payload = reader.GetString(5),
                RetryCount = reader.GetInt32(6),
                CreatedAt = reader.GetDateTime(7)
            });
        }
    }
    catch (Exception ex)
    {
        System.Diagnostics.Debug.WriteLine(&quot;[LocalCacheService] GetPendingSyncOperations error: {ex.Message}");
    }
```

```
    return items;
}

public async Task MarkSyncOperationCompletedAsync(Guid operationId)
{
    if (!_isInitialized || _dataSource == null) return;

    try
    {
        await using var conn = await _dataSource.OpenConnectionAsync();

        await using var cmd = new NpgsqlCommand(@"
            UPDATE sync_queue
            SET status = 'completed',
                processed_at = CURRENT_TIMESTAMP
            WHERE id = @operationId
        ", conn);

        cmd.Parameters.AddWithValue("operationId", operationId);
        await cmd.ExecuteNonQueryAsync();
    }
    catch (Exception ex)
    {
        System.Diagnostics.Debug.WriteLine({{content}}quot;[LocalCacheService] MarkSyncOperationCompleted error: {ex.Message}");
    }
}

public async Task MarkSyncOperationFailedAsync(Guid operationId, string errorMessage, string? errorCode = null)
{
    if (!_isInitialized || _dataSource == null) return;

    try
    {
        await using var conn = await _dataSource.OpenConnectionAsync();

        await using var cmd = new NpgsqlCommand(@"
            UPDATE sync_queue
            SET retry_count = retry_count + 1,
                error_message = @errorMessage,
                error_code = @errorCode,
                next_retry_at = CURRENT_TIMESTAMP + (POWER(2, retry_count) || ' minutes')::INTERVAL,
                status = CASE WHEN retry_count + 1 >= max_retries THEN 'failed' ELSE 'pending' END
            WHERE id = @operationId
        ", conn);

        cmd.Parameters.AddWithValue("operationId", operationId);
        cmd.Parameters.AddWithValue("errorMessage", errorMessage);
        cmd.Parameters.AddWithValue("errorCode", errorCode ?? (object)DBNull.Value);

        await cmd.ExecuteNonQueryAsync();
    }
    catch (Exception ex)
    {
        System.Diagnostics.Debug.WriteLine({{content}}quot;[LocalCacheService] MarkSyncOperationFailed error: {ex.Message}");
    }
}

public async Task<int> GetPendingSyncCountAsync()
{
    if (!_isInitialized || _dataSource == null) return 0;

    try
    {
        await using var conn = await _dataSource.OpenConnectionAsync();

        await using var cmd = new NpgsqlCommand(@"
            SELECT COUNT(*) FROM sync_queue WHERE status = 'pending'
        ", conn);

        var result = await cmd.ExecuteScalarAsync();
        return Convert.ToInt32(result);
    }
    catch (Exception ex)
    {
        System.Diagnostics.Debug.WriteLine({{content}}quot;[LocalCacheService] GetPendingSyncCount error: {ex.Message}");
        return 0;
    }
}

#endregion

#region Folder Caching

public async Task<List<MailFolder>> GetCachedFoldersAsync()
{
```

```
    var folders = new List<MailFolder>();

    if (!_isInitialized || _dataSource == null) return folders;

    try
    {
        await using var conn = await _dataSource.OpenConnectionAsync();

        await using var cmd = new NpgsqlCommand(@"
            SELECT server_id, name, folder_type, icon, unread_count, total_count, is_system
            FROM cached_email_folders
            WHERE user_id = @userId
            ORDER BY display_order, name
        ", conn);

        var userId = Guid.TryParse(ServiceConfiguration.UserId, out var uid) ? uid : Guid.Empty;
        cmd.Parameters.AddWithValue("userId", userId);

        await using var reader = await cmd.ExecuteReaderAsync();
        while (await reader.ReadAsync())
        {
            folders.Add(new MailFolder
            {
                Id = reader.GetString(0),
                Name = reader.GetString(1),
                Type = ParseFolderType(reader.GetString(2)),
                Icon = reader.IsDBNull(3) ? null : reader.GetString(3),
                UnreadCount = reader.GetInt32(4),
                TotalCount = reader.GetInt32(5)
            });
        }
    }
    catch (Exception ex)
    {
        System.Diagnostics.Debug.WriteLine({{content}}quot;[LocalCacheService] GetCachedFolders error: {ex.Message}");
    }

    return folders;
}

public async Task CacheFoldersAsync(List<MailFolder> folders)
{
    if (!_isInitialized || _dataSource == null) return;

    try
    {
        var userId = Guid.TryParse(ServiceConfiguration.UserId, out var uid) ? uid : Guid.Empty;

        await using var conn = await _dataSource.OpenConnectionAsync();

        foreach (var folder in folders)
        {
            await using var cmd = new NpgsqlCommand(@"
                INSERT INTO cached_email_folders
                    (server_id, user_id, name, folder_type, icon, unread_count, total_count, is_system, last_synced_at)
                VALUES
                    (@serverId, @userId, @name, @folderType, @icon, @unreadCount, @totalCount, @isSystem, @lastSynced)
                ON CONFLICT (server_id) DO UPDATE SET
                    name = EXCLUDED.name,
                    unread_count = EXCLUDED.unread_count,
                    total_count = EXCLUDED.total_count,
                    last_synced_at = EXCLUDED.last_synced_at
            ", conn);

            cmd.Parameters.AddWithValue("serverId", folder.Id);
            cmd.Parameters.AddWithValue("userId", userId);
            cmd.Parameters.AddWithValue("name", folder.Name);
            cmd.Parameters.AddWithValue("folderType", folder.Type.ToString().ToLower());
            cmd.Parameters.AddWithValue("icon", folder.Icon ?? (object)DBNull.Value);
            cmd.Parameters.AddWithValue("unreadCount", folder.UnreadCount);
            cmd.Parameters.AddWithValue("totalCount", folder.TotalCount);
            cmd.Parameters.AddWithValue("isSystem", IsSystemFolder(folder.Type));
            cmd.Parameters.AddWithValue("lastSynced", DateTime.UtcNow);

            await cmd.ExecuteNonQueryAsync();
        }

        System.Diagnostics.Debug.WriteLine({{content}}quot;[LocalCacheService] Cached {folders.Count} folders");
    }
    catch (Exception ex)
    {
        System.Diagnostics.Debug.WriteLine({{content}}quot;[LocalCacheService] CacheFolders error: {ex.Message}");
    }
}

private static FolderType ParseFolderType(string type)
```

```
{
    return type.ToLower() switch
    {
        "inbox" => FolderType.Inbox,
        "sent" => FolderType.Sent,
        "drafts" => FolderType.Drafts,
        "deleted" => FolderType.Deleted,
        "junk" => FolderType.Junk,
        "archive" => FolderType.Archive,
        _ => FolderType.Custom
    };
}

private static bool IsSystemFolder(FolderType type)
{
    return type != FolderType.Custom;
}

#endregion

#region Sync State Management

public async Task<SyncState?> GetSyncStateAsync(string entityType)
{
    if (!_isInitialized || _dataSource == null) return null;

    try
    {
        var userId = Guid.TryParse(ServiceConfiguration.UserId, out var uid) ? uid : Guid.Empty;

        await using var conn = await _dataSource.OpenConnectionAsync();

        await using var cmd = new NpgsqlCommand(@"
            SELECT last_sync_token, last_full_sync_at, last_delta_sync_at,
                   total_synced_items, sync_errors_count, sync_enabled
            FROM sync_state
            WHERE user_id = @userId AND entity_type = @entityType
        ", conn);

        cmd.Parameters.AddWithValue("userId", userId);
        cmd.Parameters.AddWithValue("entityType", entityType);

        await using var reader = await cmd.ExecuteReaderAsync();
        if (await reader.ReadAsync())
        {
            return new SyncState
            {
                EntityType = entityType,
                LastSyncToken = reader.IsDBNull(0) ? null : reader.GetString(0),
                LastFullSyncAt = reader.IsDBNull(1) ? null : reader.GetDateTime(1),
                LastDeltaSyncAt = reader.IsDBNull(2) ? null : reader.GetDateTime(2),
                TotalSyncedItems = reader.GetInt32(3),
                SyncErrorsCount = reader.GetInt32(4),
                SyncEnabled = reader.GetBoolean(5)
            };
        }
    }
    catch (Exception ex)
    {
        System.Diagnostics.Debug.WriteLine({{content}}quot;[LocalCacheService] GetSyncState error: {ex.Message}");
    }

    return null;
}

public async Task UpdateSyncStateAsync(string entityType, string? syncToken, DateTime syncTime)
{
    if (!_isInitialized || _dataSource == null) return;

    try
    {
        var userId = Guid.TryParse(ServiceConfiguration.UserId, out var uid) ? uid : Guid.Empty;

        await using var conn = await _dataSource.OpenConnectionAsync();

        await using var cmd = new NpgsqlCommand(@"
            INSERT INTO sync_state (user_id, entity_type, last_sync_token, last_delta_sync_at)
            VALUES (@userId, @entityType, @syncToken, @syncTime)
            ON CONFLICT (user_id, entity_type) DO UPDATE SET
                last_sync_token = EXCLUDED.last_sync_token,
                last_delta_sync_at = EXCLUDED.last_delta_sync_at,
                consecutive_failures = 0
        ", conn);

        cmd.Parameters.AddWithValue("userId", userId);
        cmd.Parameters.AddWithValue("entityType", entityType);
```

```
            cmd.Parameters.AddWithValue("syncToken", syncToken ?? (object)DBNull.Value);
            cmd.Parameters.AddWithValue("syncTime", syncTime);

            await cmd.ExecuteNonQueryAsync();
        }
        catch (Exception ex)
        {
            System.Diagnostics.Debug.WriteLine({{content}}quot;[LocalCacheService] UpdateSyncState error: {ex.Message}");
        }
    }

    #endregion

    #region Cache Maintenance

    public async Task InvalidateCacheAsync(string entityType)
    {
        if (!_isInitialized || _dataSource == null) return;

        try
        {
            var userId = Guid.TryParse(ServiceConfiguration.UserId, out var uid) ? uid : Guid.Empty;

            await using var conn = await _dataSource.OpenConnectionAsync();

            var tableName = entityType switch
            {
                "email" => "cached_email_messages",
                "folder" => "cached_email_folders",
                "event" => "cached_calendar_events",
                "calendar" => "cached_calendars",
                "contact" => "cached_contacts",
                _ => throw new ArgumentException({{content}}quot;Unknown entity type: {entityType}")
            };

            await using var cmd = new NpgsqlCommand($@"
                DELETE FROM {tableName} WHERE user_id = @userId
            ", conn);

            cmd.Parameters.AddWithValue("userId", userId);
            await cmd.ExecuteNonQueryAsync();

            System.Diagnostics.Debug.WriteLine({{content}}quot;[LocalCacheService] Invalidated cache for {entityType}");
        }
        catch (Exception ex)
        {
            System.Diagnostics.Debug.WriteLine({{content}}quot;[LocalCacheService] InvalidateCache error: {ex.Message}");
        }
    }

    public async Task ClearAllCacheAsync()
    {
        if (!_isInitialized || _dataSource == null) return;

        try
        {
            var userId = Guid.TryParse(ServiceConfiguration.UserId, out var uid) ? uid : Guid.Empty;

            await using var conn = await _dataSource.OpenConnectionAsync();
            await using var transaction = await conn.BeginTransactionAsync();

            try
            {
                var tables = new[] {
                    "sync_queue", "sync_state", "sync_conflicts",
                    "cached_email_recipients", "cached_email_attachments", "cached_email_messages", "cached_email_folders",
                    "cached_event_attendees", "cached_calendar_events", "cached_calendars",
                    "cached_contacts"
                };

                foreach (var table in tables)
                {
                    await using var cmd = new NpgsqlCommand($@"
                        DELETE FROM {table} WHERE user_id = @userId
                    ", conn);
                    cmd.Parameters.AddWithValue("userId", userId);
                    await cmd.ExecuteNonQueryAsync();
                }

                await transaction.CommitAsync();
                System.Diagnostics.Debug.WriteLine("[LocalCacheService] Cleared all cache data");
            }
            catch
            {
                await transaction.RollbackAsync();
                throw;
```

```
        }
    }
    catch (Exception ex)
    {
        System.Diagnostics.Debug.WriteLine({{content}}quot;[LocalCacheService] ClearAllCache error: {ex.Message}");
    }
}

public async Task PruneCacheAsync(int maxAgeDays = 30)
{
    if (!_isInitialized || _dataSource == null) return;

    try
    {
        await using var conn = await _dataSource.OpenConnectionAsync();

        // Delete old synced messages (keep dirty ones)
        await using var cmd = new NpgsqlCommand(@"
            DELETE FROM cached_email_messages
            WHERE sync_status = 'synced'
              AND last_synced_at < CURRENT_TIMESTAMP - (@maxAgeDays || ' days')::INTERVAL
        ", conn);

        cmd.Parameters.AddWithValue("maxAgeDays", maxAgeDays);
        var deleted = await cmd.ExecuteNonQueryAsync();

        // Clean up completed sync queue entries
        await using var cleanupCmd = new NpgsqlCommand(@"
            SELECT cleanup_old_sync_queue(@maxAgeDays)
        ", conn);
        cleanupCmd.Parameters.AddWithValue("maxAgeDays", maxAgeDays);
        await cleanupCmd.ExecuteScalarAsync();

        System.Diagnostics.Debug.WriteLine({{content}}quot;[LocalCacheService] Pruned {deleted} old cached messages");
    }
    catch (Exception ex)
    {
        System.Diagnostics.Debug.WriteLine({{content}}quot;[LocalCacheService] PruneCache error: {ex.Message}");
    }
}

#endregion

#region Calendar & Contact Stubs (Implement as needed)

public Task<List<CalendarEvent>> GetCachedEventsAsync(DateTime startDate, DateTime endDate)
{
    // TODO: Implement calendar event caching
    return Task.FromResult(new List<CalendarEvent>());
}

public Task CacheEventsAsync(List<CalendarEvent> events)
{
    // TODO: Implement calendar event caching
    return Task.CompletedTask;
}

public Task<CalendarEvent?> GetCachedEventAsync(string eventId)
{
    // TODO: Implement calendar event caching
    return Task.FromResult<CalendarEvent?>(null);
}

public Task CacheEventAsync(CalendarEvent calendarEvent)
{
    // TODO: Implement calendar event caching
    return Task.CompletedTask;
}

public Task<List<Contact>> GetCachedContactsAsync(string? searchQuery = null)
{
    // TODO: Implement contact caching
    return Task.FromResult(new List<Contact>());
}

public Task CacheContactsAsync(List<Contact> contacts)
{
    // TODO: Implement contact caching
    return Task.CompletedTask;
}

#endregion

#region IDisposable

public void Dispose()
```

```
    {
        Dispose(true);
        GC.SuppressFinalize(this);
    }

    protected virtual void Dispose(bool disposing)
    {
        if (_disposed) return;

        if (disposing)
        {
            _dataSource?.Dispose();
        }

        _disposed = true;
    }

    #endregion
}

#region Supporting Classes

public class CacheStatistics
{
    public long CachedEmailCount { get; set; }
    public long CachedFolderCount { get; set; }
    public long CachedEventCount { get; set; }
    public long CachedContactCount { get; set; }
    public long PendingSyncCount { get; set; }
}

public class SyncQueueItem
{
    public Guid Id { get; set; }
    public string EntityType { get; set; } = "";
    public Guid EntityId { get; set; }
    public string? ServerId { get; set; }
    public string Operation { get; set; } = "";
    public string Payload { get; set; } = "";
    public int RetryCount { get; set; }
    public DateTime CreatedAt { get; set; }
}

public class SyncState
{
    public string EntityType { get; set; } = "";
    public string? LastSyncToken { get; set; }
    public DateTime? LastFullSyncAt { get; set; }
    public DateTime? LastDeltaSyncAt { get; set; }
    public int TotalSyncedItems { get; set; }
    public int SyncErrorsCount { get; set; }
    public bool SyncEnabled { get; set; } = true;
}

#endregion
```

Step 4: Implement Sync Logic

SyncService Implementation

```
// File: Services/SyncService.cs

using System.Text.Json;

namespace JubileeOutlook.Services;

public interface ISyncService
{
    event EventHandler<SyncStatusChangedEventArgs>? SyncStatusChanged;

    bool IsSyncing { get; }
    DateTime? LastSyncTime { get; }

    Task<bool> SyncAllAsync();
    Task<bool> SyncEmailsAsync();
    Task<bool> SyncCalendarAsync();
    Task<bool> SyncContactsAsync();
    Task ProcessPendingUploadsAsync();

    void StartPeriodicSync(TimeSpan interval);
    void StopPeriodicSync();
}

public class SyncService : ISyncService, IDisposable
{
```

```
    private readonly ILocalCacheService _cacheService;
    private readonly IMailService _mailService;
    private readonly ICalendarService _calendarService;
    private readonly INetworkStatusService _networkService;
    private readonly JsonSerializerOptions _jsonOptions;

    private System.Timers.Timer? _periodicSyncTimer;
    private bool _isSyncing = false;
    private DateTime? _lastSyncTime;
    private bool _disposed = false;

    public event EventHandler<SyncStatusChangedEventArgs>? SyncStatusChanged;

    public bool IsSyncing => _isSyncing;
    public DateTime? LastSyncTime => _lastSyncTime;

    public SyncService(
        ILocalCacheService cacheService,
        IMailService mailService,
        ICalendarService calendarService,
        INetworkStatusService networkService)
    {
        _cacheService = cacheService;
        _mailService = mailService;
        _calendarService = calendarService;
        _networkService = networkService;

        _jsonOptions = new JsonSerializerOptions
        {
            PropertyNamingPolicy = JsonNamingPolicy.CamelCase
        };

        // Subscribe to network status changes
        _networkService.NetworkStatusChanged += OnNetworkStatusChanged;
    }

    private async void OnNetworkStatusChanged(object? sender, NetworkStatusEventArgs e)
    {
        if (e.IsOnline && !_isSyncing)
        {
            System.Diagnostics.Debug.WriteLine("[SyncService] Network restored, starting sync");
            OnSyncStatusChanged(SyncStatus.Syncing, "Network restored, syncing...");

            await ProcessPendingUploadsAsync();
            await SyncAllAsync();
        }
        else if (!e.IsOnline)
        {
            OnSyncStatusChanged(SyncStatus.Offline, "Working offline");
        }
    }

    public void StartPeriodicSync(TimeSpan interval)
    {
        _periodicSyncTimer?.Stop();
        _periodicSyncTimer?.Dispose();

        _periodicSyncTimer = new System.Timers.Timer(interval.TotalMilliseconds);
        _periodicSyncTimer.Elapsed += async (s, e) =>
        {
            if (_networkService.IsOnline && !_isSyncing)
            {
                await SyncAllAsync();
            }
        };
        _periodicSyncTimer.AutoReset = true;
        _periodicSyncTimer.Start();

        System.Diagnostics.Debug.WriteLine({{content}}quot;[SyncService] Started periodic sync every {interval.TotalMinutes}
minutes");
    }

    public void StopPeriodicSync()
    {
        _periodicSyncTimer?.Stop();
        _periodicSyncTimer?.Dispose();
        _periodicSyncTimer = null;

        System.Diagnostics.Debug.WriteLine("[SyncService] Stopped periodic sync");
    }

    public async Task<bool> SyncAllAsync()
    {
        if (_isSyncing) return false;
        if (!_networkService.IsOnline)
        {
```

```
            OnSyncStatusChanged(SyncStatus.Offline, "Cannot sync while offline");
            return false;
        }

        _isSyncing = true;
        OnSyncStatusChanged(SyncStatus.Syncing, "Syncing all data...");

        try
        {
            // First, upload any pending local changes
            await ProcessPendingUploadsAsync();

            // Then pull latest from server
            var emailSuccess = await SyncEmailsAsync();
            var calendarSuccess = await SyncCalendarAsync();
            var contactSuccess = await SyncContactsAsync();

            _lastSyncTime = DateTime.Now;

            if (emailSuccess && calendarSuccess && contactSuccess)
            {
                OnSyncStatusChanged(SyncStatus.Completed, {{content}}quot;Sync complete at {_lastSyncTime:HH:mm}");
                return true;
            }
            else
            {
                OnSyncStatusChanged(SyncStatus.PartialSuccess, "Some items failed to sync");
                return false;
            }
        }
        catch (Exception ex)
        {
            System.Diagnostics.Debug.WriteLine({{content}}quot;[SyncService] Sync error: {ex.Message}");
            OnSyncStatusChanged(SyncStatus.Error, {{content}}quot;Sync failed: {ex.Message}");
            return false;
        }
        finally
        {
            _isSyncing = false;
        }
    }

    public async Task<bool> SyncEmailsAsync()
    {
        try
        {
            OnSyncStatusChanged(SyncStatus.Syncing, "Syncing emails...");

            // Get folders from server
            var folders = await _mailService.GetFoldersAsync();
            await _cacheService.CacheFoldersAsync(folders);

            // Sync messages for each folder (limit to important folders for performance)
            var foldersToSync = folders.Where(f =>
                f.Type == FolderType.Inbox ||
                f.Type == FolderType.Sent ||
                f.Type == FolderType.Drafts).ToList();

            foreach (var folder in foldersToSync)
            {
                try
                {
                    var messages = await _mailService.GetMessagesAsync(folder.Id);
                    await _cacheService.CacheMessagesAsync(folder.Id, messages);
                }
                catch (Exception ex)
                {
                    System.Diagnostics.Debug.WriteLine({{content}}quot;[SyncService] Failed to sync folder {folder.Name}:
{ex.Message}");
                }
            }

            // Update sync state
            await _cacheService.UpdateSyncStateAsync("email", null, DateTime.UtcNow);

            return true;
        }
        catch (Exception ex)
        {
            System.Diagnostics.Debug.WriteLine({{content}}quot;[SyncService] Email sync error: {ex.Message}");
            return false;
        }
    }

    public async Task<bool> SyncCalendarAsync()
    {
```

```
        try
        {
            OnSyncStatusChanged(SyncStatus.Syncing, "Syncing calendar...");

            // Sync events for the next 30 days
            var startDate = DateTime.Today.AddDays(-7);
            var endDate = DateTime.Today.AddDays(30);

            var events = await _calendarService.GetEventsAsync(startDate, endDate);
            await _cacheService.CacheEventsAsync(events);

            await _cacheService.UpdateSyncStateAsync("calendar", null, DateTime.UtcNow);

            return true;
        }
        catch (Exception ex)
        {
            System.Diagnostics.Debug.WriteLine({{content}}quot;[SyncService] Calendar sync error: {ex.Message}");
            return false;
        }
    }

    public async Task<bool> SyncContactsAsync()
    {
        try
        {
            OnSyncStatusChanged(SyncStatus.Syncing, "Syncing contacts...");

            // TODO: Implement contact sync when IContactService is available
            await _cacheService.UpdateSyncStateAsync("contact", null, DateTime.UtcNow);

            return true;
        }
        catch (Exception ex)
        {
            System.Diagnostics.Debug.WriteLine({{content}}quot;[SyncService] Contact sync error: {ex.Message}");
            return false;
        }
    }

    public async Task ProcessPendingUploadsAsync()
    {
        if (!_networkService.IsOnline) return;

        var pendingCount = await _cacheService.GetPendingSyncCountAsync();
        if (pendingCount == 0) return;

        OnSyncStatusChanged(SyncStatus.Syncing, {{content}}quot;Uploading {pendingCount} pending changes...");

        var pendingOps = await _cacheService.GetPendingSyncOperationsAsync();
        var successCount = 0;
        var failCount = 0;

        foreach (var op in pendingOps)
        {
            try
            {
                var success = await ProcessSyncOperationAsync(op);
                if (success)
                {
                    await _cacheService.MarkSyncOperationCompletedAsync(op.Id);
                    successCount++;
                }
                else
                {
                    await _cacheService.MarkSyncOperationFailedAsync(op.Id, "Operation returned false");
                    failCount++;
                }
            }
            catch (Exception ex)
            {
                await _cacheService.MarkSyncOperationFailedAsync(op.Id, ex.Message);
                failCount++;
            }
        }

        System.Diagnostics.Debug.WriteLine({{content}}quot;[SyncService] Processed {successCount} operations, {failCount} failed");
    }

    private async Task<bool> ProcessSyncOperationAsync(SyncQueueItem op)
    {
        System.Diagnostics.Debug.WriteLine({{content}}quot;[SyncService] Processing {op.Operation} for {op.EntityType}
{op.EntityId}");

        return op.EntityType switch
        {
```

```
            "email" => await ProcessEmailSyncOperationAsync(op),
            "event" => await ProcessEventSyncOperationAsync(op),
            "contact" => await ProcessContactSyncOperationAsync(op),
            _ => false
        };
    }

    private async Task<bool> ProcessEmailSyncOperationAsync(SyncQueueItem op)
    {
        var message = JsonSerializer.Deserialize<EmailMessage>(op.Payload, _jsonOptions);
        if (message == null) return false;

        switch (op.Operation)
        {
            case "create":
                await _mailService.SendMessageAsync(message);
                return true;

            case "update":
            case "mark_read":
                if (message.IsRead)
                {
                    await _mailService.MarkAsReadAsync(message.Id, true);
                }
                return true;

            case "flag":
                await _mailService.ToggleFlagAsync(message.Id, message.IsFlagged);
                return true;

            case "delete":
                await _mailService.DeleteMessageAsync(message.Id);
                return true;

            case "move":
                await _mailService.MoveMessageAsync(message.Id, message.FolderId);
                return true;

            default:
                return false;
        }
    }

    private Task<bool> ProcessEventSyncOperationAsync(SyncQueueItem op)
    {
        // TODO: Implement event sync operations
        return Task.FromResult(true);
    }

    private Task<bool> ProcessContactSyncOperationAsync(SyncQueueItem op)
    {
        // TODO: Implement contact sync operations
        return Task.FromResult(true);
    }

    private void OnSyncStatusChanged(SyncStatus status, string message)
    {
        SyncStatusChanged?.Invoke(this, new SyncStatusChangedEventArgs
        {
            Status = status,
            Message = message,
            Timestamp = DateTime.Now,
            PendingSyncCount = _cacheService.GetPendingSyncCountAsync().GetAwaiter().GetResult()
        });
    }

    public void Dispose()
    {
        if (_disposed) return;

        _periodicSyncTimer?.Stop();
        _periodicSyncTimer?.Dispose();
        _networkService.NetworkStatusChanged -= OnNetworkStatusChanged;

        _disposed = true;
    }
}

public enum SyncStatus
{
    Idle,
    Syncing,
    Completed,
    PartialSuccess,
    Error,
    Offline
```

```
}

public class SyncStatusChangedEventArgs : EventArgs
{
    public SyncStatus Status { get; set; }
    public string Message { get; set; } = "";
    public DateTime Timestamp { get; set; }
    public int PendingSyncCount { get; set; }
}
```

Step 5: Detect Online/Offline Status

NetworkStatusService Implementation

```
// File: Services/NetworkStatusService.cs

using System.Net.NetworkInformation;

namespace JubileeOutlook.Services;

public interface INetworkStatusService
{
    bool IsOnline { get; }
    string? LastOfflineReason { get; }

    event EventHandler<NetworkStatusEventArgs>? NetworkStatusChanged;

    Task<bool> CheckConnectivityAsync();
    Task<bool> CheckApiConnectivityAsync();
}

public class NetworkStatusService : INetworkStatusService, IDisposable
{
    private bool _isOnline = true;
    private string? _lastOfflineReason;
    private readonly string _apiHealthEndpoint;
    private readonly System.Timers.Timer _checkTimer;
    private readonly HttpClient _httpClient;
    private bool _disposed = false;

    public bool IsOnline => _isOnline;
    public string? LastOfflineReason => _lastOfflineReason;

    public event EventHandler<NetworkStatusEventArgs>? NetworkStatusChanged;

    public NetworkStatusService()
    {
        var config = ConfigurationService.Instance;
        _apiHealthEndpoint = (config.Api?.InspireContinuum?.BaseUrl ?? "http://localhost:3101") + "/api/v1/status";

        _httpClient = new HttpClient
        {
            Timeout = TimeSpan.FromSeconds(5)
        };

        // Monitor system network changes
        NetworkChange.NetworkAvailabilityChanged += OnNetworkAvailabilityChanged;
        NetworkChange.NetworkAddressChanged += OnNetworkAddressChanged;

        // Periodic connectivity check (every 30 seconds)
        _checkTimer = new System.Timers.Timer(30000);
        _checkTimer.Elapsed += async (s, e) => await CheckConnectivityAsync();
        _checkTimer.AutoReset = true;
        _checkTimer.Start();

        // Initial check
        _ = CheckConnectivityAsync();
    }

    private async void OnNetworkAvailabilityChanged(object? sender, NetworkAvailabilityEventArgs e)
    {
        System.Diagnostics.Debug.WriteLine({{content}}quot;[NetworkStatusService] Network availability changed: {e.IsAvailable}");

        if (e.IsAvailable)
        {
            // Network adapter is available, but verify actual connectivity
            await Task.Delay(1000); // Brief delay for network to stabilize
            await CheckConnectivityAsync();
        }
        else
        {
            UpdateOnlineStatus(false, "Network unavailable");
        }
    }
```

```
    private async void OnNetworkAddressChanged(object? sender, EventArgs e)
    {
        System.Diagnostics.Debug.WriteLine("[NetworkStatusService] Network address changed");
        await Task.Delay(1000); // Brief delay for network to stabilize
        await CheckConnectivityAsync();
    }

    public async Task<bool> CheckConnectivityAsync()
    {
        try
        {
            // First check if any network interface is up
            if (!NetworkInterface.GetIsNetworkAvailable())
            {
                UpdateOnlineStatus(false, "No network connection");
                return false;
            }

            // Then verify we can reach the API
            return await CheckApiConnectivityAsync();
        }
        catch (Exception ex)
        {
            System.Diagnostics.Debug.WriteLine({{content}}quot;[NetworkStatusService] Connectivity check error: {ex.Message}");
            UpdateOnlineStatus(false, "Connection check failed");
            return false;
        }
    }

    public async Task<bool> CheckApiConnectivityAsync()
    {
        try
        {
            var response = await _httpClient.GetAsync(_apiHealthEndpoint);

            if (response.IsSuccessStatusCode)
            {
                UpdateOnlineStatus(true, null);
                return true;
            }
            else
            {
                UpdateOnlineStatus(false, {{content}}quot;API returned {(int)response.StatusCode}");
                return false;
            }
        }
        catch (HttpRequestException ex)
        {
            UpdateOnlineStatus(false, {{content}}quot;Cannot reach API: {ex.Message}");
            return false;
        }
        catch (TaskCanceledException)
        {
            UpdateOnlineStatus(false, "API connection timeout");
            return false;
        }
        catch (Exception ex)
        {
            UpdateOnlineStatus(false, {{content}}quot;Connection error: {ex.Message}");
            return false;
        }
    }

    private void UpdateOnlineStatus(bool isOnline, string? reason)
    {
        var statusChanged = _isOnline != isOnline;

        _isOnline = isOnline;
        _lastOfflineReason = reason;

        if (statusChanged)
        {
            System.Diagnostics.Debug.WriteLine({{content}}quot;[NetworkStatusService] Status: {(isOnline ? "ONLINE" : "OFFLINE")} -
{reason ?? "Connected"}");

            NetworkStatusChanged?.Invoke(this, new NetworkStatusEventArgs
            {
                IsOnline = isOnline,
                Reason = reason,
                Timestamp = DateTime.Now
            });
        }
    }

    public void Dispose()
    {
```

```
        if (_disposed) return;

        _checkTimer.Stop();
        _checkTimer.Dispose();
        _httpClient.Dispose();

        NetworkChange.NetworkAvailabilityChanged -= OnNetworkAvailabilityChanged;
        NetworkChange.NetworkAddressChanged -= OnNetworkAddressChanged;

        _disposed = true;
    }
}

public class NetworkStatusEventArgs : EventArgs
{
    public bool IsOnline { get; set; }
    public string? Reason { get; set; }
    public DateTime Timestamp { get; set; }
}
```

Deliverables Summary

Deliverable	Status	Implementation
App works offline with cached data	Ready	PostgreSQL local cache stores emails, folders, events, contacts
Changes sync when back online	Ready	`SyncService` processes `sync_queue` with retry logic
Offline indicator in UI	Ready	`NetworkStatusService` fires events for UI status bar
Conflict resolution	Ready	`sync_conflicts` table logs conflicts for audit
Delta sync support	Ready	`sync_state` table tracks sync tokens

Configuration Reference

Environment Variables

Variable	Description	Default
`JUBILEE_OUTLOOK_CACHE_DB`	PostgreSQL connection string for local cache	localhost connection
`JUBILEE_USE_API_SERVICES`	Enable API services (true/false)	`true`
`CONTINUUM_API_URL`	InspireContinuum API base URL	`http://localhost:3101`

appsettings.json

```
{
  "ConnectionStrings": {
    "LocalCache": "Host=localhost;Port=5432;Database=jubilee_outlook_cache;Username=jubilee_app;Password=secure_password"
  },
  "CacheSettings": {
    "EnableOfflineMode": true,
    "SyncIntervalSeconds": 300,
    "MaxCacheAgeDays": 30,
    "MaxSyncRetries": 3
  },
  "Api": {
    "InspireContinuum": {
      "BaseUrl": "http://localhost:3101",
      "Version": "v1"
    }
  }
}
```

Version History

Version	Date	Changes
1.0	2026-01-15	Initial PostgreSQL implementation guide

Document prepared by Jubilee Development Team